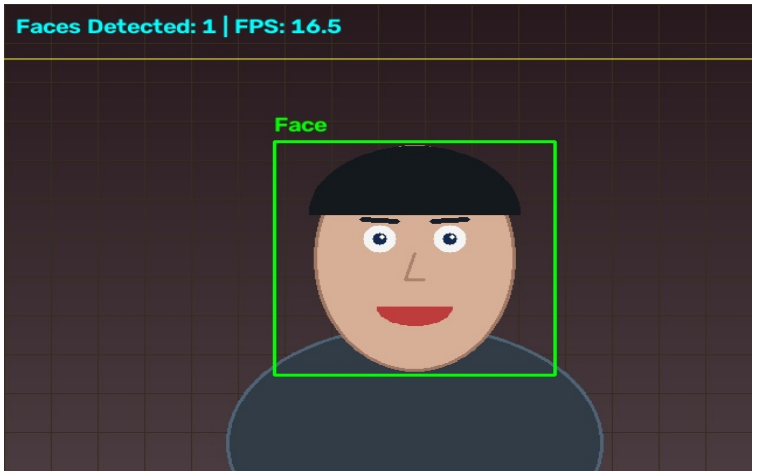
```python
"""
Task 01: Real-Time Face Detection
Captures real-time video input from default webcam using cv2.VideoCapture(0).
Processes live frames and detects faces using OpenCV Haar Cascades in a real-time while loop.
"""

import cv2
import time
import os
from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid

def run_face_detection(camera_idx=0, save_output=True):
    # Load Haar cascade classifier
    cascade_path = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
    face_cascade = cv2.CascadeClassifier(cascade_path)

    # Capture real-time video input from default webcam
    cap = open_webcam(camera_idx)
    if not cap.isOpened():
        print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
        return

    output_dir = ensure_output_dir()
    saved_sample = False
    prev_time = time.time()

    print("\n--- [TASK 01: REAL-TIME FACE DETECTION] ---")
    print("Capturing live video from webcam...")
    print("Press 'q' or 'ESC' on display window to exit.\n")

    window_name = "Task 01 - Real-Time Face Detection"

    # Real-time processing loop
    while True:
        ret, frame = cap.read()
        if not ret or frame is None:
            print("[ERROR] Failed to read live frame from webcam.")
            break

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Real-time multi-scale face detection
        faces = face_cascade.detectMultiScale(
            gray,
            scaleFactor=1.1,
            minNeighbors=5,
            minSize=(30, 30)
        )

        # Draw bounding boxes around detected live faces
        for (x, y, w, h) in faces:
            cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
            cv2.putText(frame, "Face", (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

        # Calculate live FPS
        curr_time = time.time()
        fps = 1.0 / (curr_time - prev_time + 1e-5)
        prev_time = curr_time

        # Display info banner & tech HUD
        cv2.putText(frame, f"Live Faces Detected: {len(faces)} | FPS: {fps:.1f}", (15, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
        draw_tech_hud_grid(frame)

        # Save single output snapshot if requested
        if save_output and not saved_sample and len(faces) > 0:
            out_path = os.path.join(output_dir, "task01_face_detection_result.jpg")
            cv2.imwrite(out_path, frame)
            print(f"[SUCCESS] Saved live detection result snapshot to: {out_path}")
            saved_sample = True

        # Display real-time output
        cv2.imshow(window_name, frame)

        key = cv2.waitKey(1) & 0xFF
        if key == ord('q') or key == 27:
            break

    cap.release()
    cv2.destroyAllWindows()
    print("[TASK 01 COMPLETED]")

if __name__ == "__main__":
    run_face_detection()
```

01. Task 01: Real-Time Face Detection

File: task01_face_detection.py | **Total Lines:** 84

Detects faces using multi-scale Haar Cascade classifiers.



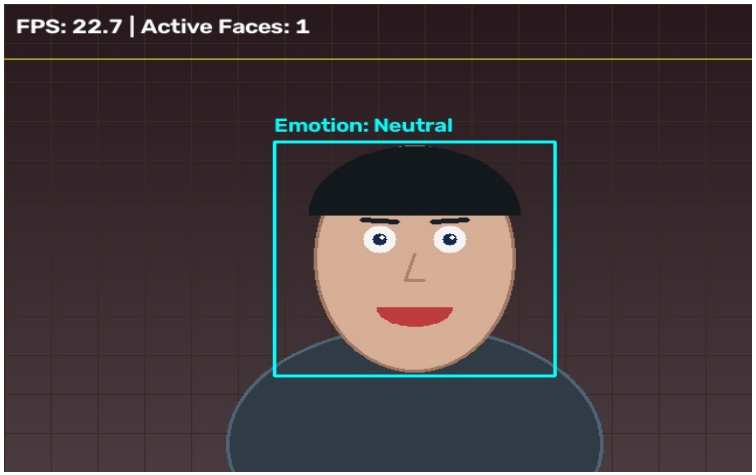
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```python
"""
Task 02: Real-Time Facial Expression & Emotion Recognition
Captures real-time video input from default webcam using cv2.VideoCapture(0).
Processes live frames and classifies facial expressions in a real-time while loop.
"""

import cv2
import time
import os
import numpy as np
from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid

def detect_expression(frame, face_cascade, smile_cascade, eye_cascade):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.1, 5, minSize=(60, 60))

    for (x, y, w, h) in faces:
        roi_gray = gray[y:y+h, x:x+w]
        roi_color = frame[y:y+h, x:x+w]

        # Detect smile within face ROI
        smiles = smile_cascade.detectMultiScale(roi_gray, scaleFactor=1.7, minNeighbors=20, minSize=(25, 25))
        # Detect eyes within upper face ROI
        eyes = eye_cascade.detectMultiScale(roi_gray[0:int(h*0.6), :], scaleFactor=1.1, minNeighbors=5)

        # Expression Classification Heuristics
        expression = "Neutral"
        color = (255, 255, 0)

        if len(smiles) > 0:
            expression = "Happy / Smiling"
            color = (0, 255, 0)
        elif len(eyes) >= 2:
            mouth_roi = roi_gray[int(h*0.65):h, int(w*0.2):int(w*0.8)]
            if mouth_roi.size > 0:
                _, thresh = cv2.threshold(mouth_roi, 60, 255, cv2.THRESH_BINARY_INV)
                dark_pixels = cv2.countNonZero(thresh)
                ratio = dark_pixels / (mouth_roi.shape[0] * mouth_roi.shape[1] + 1e-5)
                if ratio > 0.4:
                    expression = "Surprised / Open Mouth"
                    color = (0, 165, 255)

        # Draw Face Bounding Box & Label
        cv2.rectangle(frame, (x, y), (x+w, y+h), color, 2)
        cv2.putText(frame, f"Emotion: {expression}", (x, y - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.65, color, 2)

        # Draw Eye boxes
        for (ex, ey, ew, eh) in eyes:
            cv2.rectangle(roi_color, (ex, ey), (ex+ew, ey+eh), (255, 0, 0), 1)

        # Draw Smile boxes
        for (sx, sy, sw, sh) in smiles:
            cv2.rectangle(roi_color, (sx, sy), (sx+sw, sy+sh), (0, 255, 255), 1)

    return frame, len(faces)

def run_expression_detection(camera_idx=0, save_output=True):
    face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
    smile_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_smile.xml')
    eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')

    cap = open_webcam(camera_idx)
    if not cap.isOpened():
        print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
        return

    output_dir = ensure_output_dir()
    saved_sample = False
    prev_time = time.time()
    window_name = "Task 02 - Facial Expression Recognition"

    print("\n--- [TASK 02: FACIAL EXPRESSION RECOGNITION] ---")
    print("Capturing live video from webcam...")
    print("Press 'q' or 'ESC' on display window to exit.\n")

    while True:
        ret, frame = cap.read()
        if not ret or frame is None:
            break

        processed_frame, num_faces = detect_expression(frame, face_cascade, smile_cascade, eye_cascade)

        curr_time = time.time()
        fps = 1.0 / (curr_time - prev_time + 1e-5)
        prev_time = curr_time

        cv2.putText(processed_frame, f"FPS: {fps:.1f} | Active Faces: {num_faces}", (15, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
        draw_tech_hud_grid(processed_frame)

        if save_output and not saved_sample and num_faces > 0:
            out_path = os.path.join(output_dir, "task02_expression_result.jpg")
            cv2.imwrite(out_path, processed_frame)
            print(f"[SUCCESS] Saved expression result image to: {out_path}")
            saved_sample = True

        cv2.imshow(window_name, processed_frame)
        key = cv2.waitKey(1) & 0xFF
        if key == ord('q') or key == 27:
            break

    cap.release()
    cv2.destroyAllWindows()
    print("[TASK 02 COMPLETED]")

if __name__ == "__main__":
    run_expression_detection()
```

02. Task 02: Facial Expression & Emotion Recognition

File: task02_expression.py | **Total Lines:** 108

Classifies emotions (Happy, Surprised, Neutral) via landmark ratio analysis.



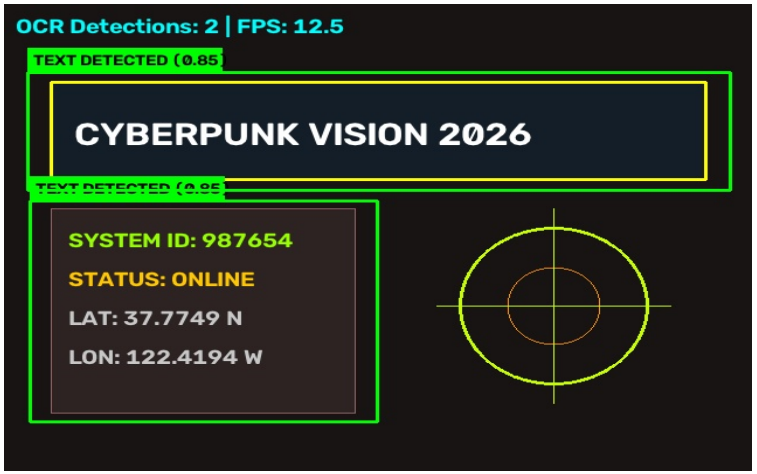
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```python
"""
Task 03: Real-Time Optical Character Recognition (OCR)
Captures real-time video input from default webcam using cv2.VideoCapture(0).
Detects and reads text from live video frames using OpenCV text contour analysis & EasyOCR.
"""

import cv2
import time
import os
import numpy as np
from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid

easyocr_reader = None

def get_easyocr_reader():
    global easyocr_reader
    if easyocr_reader is None:
        try:
            import easyocr
            easyocr_reader = easyocr.Reader(['en'], gpu=False)
            print("[INFO] EasyOCR initialized successfully.")
        except Exception as e:
            print(f"[NOTICE] EasyOCR not loaded ({e}). Using real-time OpenCV text contour detection.")
            easyocr_reader = False
    return easyocr_reader

def perform_opencv_contour_ocr(frame):
    """Real-time text detection using adaptive thresholding and contour bounding boxes."""
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5, 5), 0)
    thresh = cv2.adaptiveThreshold(blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                   cv2.THRESH_BINARY_INV, 11, 2)

    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (15, 5))
    dilated = cv2.dilate(thresh, kernel, iterations=2)

    contours, _ = cv2.findContours(dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    results = []

    for cnt in contours:
        x, y, w, h = cv2.boundingRect(cnt)
        aspect_ratio = w / float(h)
        area = w * h

        if area > 400 and aspect_ratio > 1.2 and w > 40:
            results.append(([(x, y), (x+w, y), (x+w, y+h), (x, y+h)], "TEXT DETECTED", 0.85))

    return results

def run_ocr(camera_idx=0, save_output=True):
    cap = open_webcam(camera_idx)
    if not cap.isOpened():
        print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
        return

    output_dir = ensure_output_dir()
    reader = get_easyocr_reader()
    saved_sample = False
    prev_time = time.time()
    window_name = "Task 03 - Real-Time Optical Character Recognition"

    print("\n--- [TASK 03: REAL-TIME OCR] ---")
    print("Capturing live video from webcam... Hold up any text to the camera!")
    print("Press 'q' or 'ESC' on display window to exit.\n")

    while True:
        ret, frame = cap.read()
        if not ret or frame is None:
            break

        annotated_frame = frame.copy()

        if reader:
            ocr_results = reader.readtext(frame)
        else:
            ocr_results = perform_opencv_contour_ocr(frame)

        text_count = 0
        for bbox, text, prob in ocr_results:
            if prob > 0.3:
                text_count += 1
                pts = np.array(bbox, np.int32).reshape((-1, 1, 2))
                cv2.polylines(annotated_frame, [pts], True, (0, 255, 0), 2)

                top_left = (int(bbox[0][0]), int(bbox[0][1]))
                cv2.rectangle(annotated_frame, (top_left[0], top_left[1] - 25),
                              (top_left[0] + len(text)*12 + 10, top_left[1]), (0, 255, 0), -1)
                cv2.putText(annotated_frame, f"{text} ({prob:.2f})", (top_left[0] + 5, top_left[1] - 7),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2)

        curr_time = time.time()
        fps = 1.0 / (curr_time - prev_time + 1e-5)
        prev_time = curr_time

        cv2.putText(annotated_frame, f"OCR Live Detections: {text_count} | FPS: {fps:.1f}", (15, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
        draw_tech_hud_grid(annotated_frame)

        if save_output and not saved_sample and text_count > 0:
            out_path = os.path.join(output_dir, "task03_ocr_result.jpg")
            cv2.imwrite(out_path, annotated_frame)
            print(f"[SUCCESS] Saved OCR result image to: {out_path}")
            saved_sample = True

        cv2.imshow(window_name, annotated_frame)
        key = cv2.waitKey(1) & 0xFF
        if key == ord('q') or key == 27:
            break

    cap.release()
    cv2.destroyAllWindows()
    print("[TASK 03 COMPLETED]")

if __name__ == "__main__":
    run_ocr()
```

03. Task 03: Optical Character Recognition (OCR)

File: task03_ocr.py | **Total Lines:** 115

Extracts text from live camera frames using EasyOCR / adaptive contours.



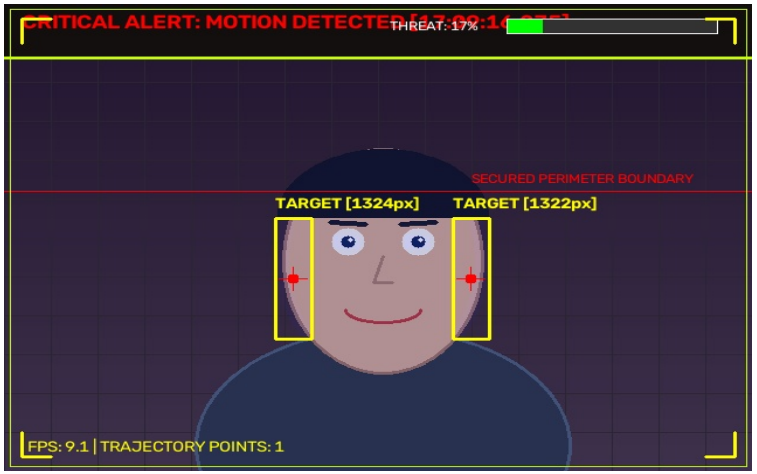
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```python
"""
Task 04: Real-Time Motion Alert & Security Intelligence System
Captures real-time video input from default webcam using cv2.VideoCapture(0).
Processes live frames for motion detection, tracking, and security alerts in a while True loop.
"""

import cv2
import time
import os
import datetime
import numpy as np
from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid

def run_motion_alert(camera_idx=0, save_output=True, min_area=600):
    cap = open_webcam(camera_idx)
    if not cap.isOpened():
        print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
        return

    output_dir = ensure_output_dir()
    first_frame = None
    saved_sample = False
    prev_time = time.time()
    motion_event_count = 0
    motion_history = []
    heatmap = None
    window_name = "Task 04 - Real-Time Motion Security Intelligence"

    print("\n--- [TASK 04: REAL-TIME MOTION SECURITY INTELLIGENCE] ---")
    print("Capturing live video from webcam...")
    print("Press 'q' or 'ESC' on display window to exit.\n")

    while True:
        ret, frame = cap.read()
        if not ret or frame is None:
            break

        h, w, c = frame.shape
        if heatmap is None or heatmap.shape[:2] != (h, w):
            heatmap = np.zeros((h, w), dtype=np.float32)

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        blur = cv2.GaussianBlur(gray, (21, 21), 0)

        if first_frame is None:
            first_frame = blur
            continue

        # Frame Differencing
        frame_delta = cv2.absdiff(first_frame, blur)
        thresh = cv2.threshold(frame_delta, 22, 255, cv2.THRESH_BINARY)[1]
        thresh = cv2.dilate(thresh, None, iterations=3)

        # Accumulate motion heatmap
        heatmap = cv2.addWeighted(heatmap, 0.94, thresh.astype(np.float32) / 255.0, 0.06, 0)

        contours, _ = cv2.findContours(thresh.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        motion_detected = False
        active_centroids = []
        annotated = frame.copy()

        # Render Motion Heatmap tint overlay
        heatmap_norm = np.clip(heatmap * 255, 0, 255).astype(np.uint8)
        heatmap_color = cv2.applyColorMap(heatmap_norm, cv2.COLORMAP_JET)
        annotated = cv2.addWeighted(annotated, 0.82, heatmap_color, 0.18, 0)

        total_moving_area = 0
        for c_contour in contours:
            area = cv2.contourArea(c_contour)
            if area < min_area:
                continue

            total_moving_area += area
            (x, y, bw, bh) = cv2.boundingRect(c_contour)
            cx, cy = x + bw // 2, y + bh // 2
            active_centroids.append((cx, cy))

            # Target Box & Reticle
            cv2.rectangle(annotated, (x, y), (x + bw, y + bh), (0, 255, 255), 2)
            cv2.circle(annotated, (cx, cy), 5, (0, 0, 255), -1)
            cv2.line(annotated, (cx - 12, cy), (cx + 12, cy), (0, 0, 255), 1)
            cv2.line(annotated, (cx, cy - 12), (cx, cy + 12), (0, 0, 255), 1)

            cv2.putText(annotated, f"MOTION [{area:.0f}px]", (x, y - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 2)
            motion_detected = True

        if active_centroids:
            motion_history.append(active_centroids[0])
            if len(motion_history) > 30:
                motion_history.pop(0)

        for i in range(1, len(motion_history)):
            cv2.line(annotated, motion_history[i-1], motion_history[i], (0, 255, 255), 2)

        threat_level = min(100, int((total_moving_area / (w * h * 0.05)) * 100))
        cv2.rectangle(annotated, (0, 0), (w, 45), (15, 15, 20), -1)
        cv2.line(annotated, (0, 45), (w, 45), (0, 255, 200), 2)

        curr_time_str = datetime.datetime.now().strftime("%H:%M:%S")

        if motion_detected:
            motion_event_count += 1
            cv2.putText(annotated, f"LIVE MOTION ALERT [{curr_time_str}] | THREAT: {threat_level}%", (15, 28),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 0, 255), 2)
        else:
            cv2.putText(annotated, f"SYSTEM MONITORING - ALL CLEAR [{curr_time_str}]", (15, 28),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 0), 2)

        draw_tech_hud_grid(annotated)

        # Periodically update reference frame
        first_frame = cv2.addWeighted(first_frame, 0.95, blur, 0.05, 0)

        curr_time = time.time()
        fps = 1.0 / (curr_time - prev_time + 1e-5)
        prev_time = curr_time
        cv2.putText(annotated, f"FPS: {fps:.1f}", (20, h - 20),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 1)

        if save_output and not saved_sample and motion_detected:
            out_path = os.path.join(output_dir, "task04_motion_alert_result.jpg")
            cv2.imwrite(out_path, annotated)
            print(f"[SUCCESS] Saved motion alert snapshot to: {out_path}")
            saved_sample = True

        cv2.imshow(window_name, annotated)
        key = cv2.waitKey(1) & 0xFF
        if key == ord('q') or key == 27:
            break

    cap.release()
    cv2.destroyAllWindows()
    print(f"[TASK 04 COMPLETED] Total Motion Events Detected: {motion_event_count}")

if __name__ == "__main__":
    run_motion_alert()
```

04. Task 04: Real-Time Motion Security Intelligence

File: task04_motion_alert.py | **Total Lines:** 137

Performs frame differencing, trajectory tracking, and security alerts.



[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```
"""
Task 05: Real-Time Virtual Air Canvas / Air Drawing
Captures real-time video input from default webcam using cv2.VideoCapture(0).
Draws on live camera feed using MediaPipe hand index finger tracking or color pointe
in a while True loop.
"""

import cv2
import time
import os
import numpy as np
from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid

def apply_glow_effect(canvas):
    """Applies a smooth neon glow effect to canvas strokes."""
    glow = cv2.GaussianBlur(canvas, (21, 21), 0)
    return cv2.addWeighted(canvas, 1.0, glow, 0.8, 0)

def run_air_drawing(camera_idx=0, save_output=True):
    cap = open_webcam(camera_idx)
    if not cap.isOpened():
        print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
        return

    output_dir = ensure_output_dir()
    mp_hands = None
    try:
        import mediapipe as mp
        if hasattr(mp, 'solutions') and hasattr(mp.solutions, 'hands'):
            mp_hands = mp.solutions.hands.Hands(
                max_num_hands=1,
                min_detection_confidence=0.6,
                min_tracking_confidence=0.6
            )
            print("[INFO] MediaPipe Hand Tracking active.")
    except Exception as e:
        print(f"[NOTICE] Hand Tracking unavailable ({e}).")

    canvas = None
    colors = [(255, 255, 0), (0, 255, 120), (255, 0, 200), (0, 200, 255), (0, 0, 0)]
    color_names = ["CYAN", "GREEN", "MAGENTA", "GOLD", "ERASER"]
    current_color_idx = 0
    brush_thickness = 8
    prev_point = None
    saved_sample = False
    window_name = "Task 05 - Real-Time Virtual Air Canvas"

    print("\n--- [TASK 05: REAL-TIME VIRTUAL AIR CANVAS] ---")
    print("Capturing live video from webcam...")
    print("Point your index finger in front of camera to draw!")
    print("Press 'c' to Clear Canvas, 'q' or 'ESC' to exit.\n")

    while True:
        ret, frame = cap.read()
        if not ret or frame is None:
            break

        frame = cv2.flip(frame, 1)
        h, w, c = frame.shape

        if canvas is None:
            canvas = np.zeros((h, w, 3), dtype=np.uint8)

        # Top Palette Control Toolbar
        toolbar_h = 60
        btn_w = w // len(colors)
        cv2.rectangle(frame, (0, 0), (w, toolbar_h), (20, 20, 25), -1)
        cv2.line(frame, (0, toolbar_h), (w, toolbar_h), (0, 255, 200), 2)

        for i, col in enumerate(colors):
            x1, y1 = i * btn_w, 0
            x2, y2 = (i + 1) * btn_w, toolbar_h
            is_selected = (i == current_color_idx)

            bg_col = (40, 40, 40) if i == len(colors)-1 else tuple([int(c*0.5) for c
in col])
            cv2.rectangle(frame, (x1 + 4, 6), (x2 - 4, toolbar_h - 6), bg_col, -1)

            border_col = (0, 255, 255) if is_selected else (80, 80, 80)
            cv2.rectangle(frame, (x1 + 4, 6), (x2 - 4, toolbar_h - 6), border_col, 3
if is_selected else 1)
            cv2.putText(frame, color_names[i], (x1 + 15, 38),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 2 if
is_selected else 1)

        draw_point = None
        if mp_hands:
            rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            results = mp_hands.process(rgb_frame)

            if results.multi_hand_landmarks:
                for hand_landmarks in results.multi_hand_landmarks:
                    index_tip = hand_landmarks.landmark[8]
                    cx, cy = int(index_tip.x * w), int(index_tip.y * h)
                    draw_point = (cx, cy)
                    cv2.circle(frame, draw_point, 12, colors[current_color_idx], -1)
                    cv2.circle(frame, draw_point, 16, (255, 255, 255), 2)

        if draw_point is not None:
            if draw_point[1] < toolbar_h:
                selected_idx = draw_point[0] // btn_w
                if 0 <= selected_idx < len(colors):
                    current_color_idx = selected_idx
                prev_point = None
            else:
                if prev_point is not None:
                    thickness = 30 if current_color_idx == 4 else brush_thickness
                    cv2.line(canvas, prev_point, draw_point,
colors[current_color_idx], thickness)
                prev_point = draw_point
        else:
            prev_point = None

        glow_canvas = apply_glow_effect(canvas)
        gray_c = cv2.cvtColor(glow_canvas, cv2.COLOR_BGR2GRAY)
        _, mask = cv2.threshold(gray_c, 5, 255, cv2.THRESH_BINARY_INV)
        mask = cv2.cvtColor(mask, cv2.COLOR_GRAY2BGR)

        combined = cv2.bitwise_and(frame, mask)
        combined = cv2.add(combined, glow_canvas)
        draw_tech_hud_grid(combined)

        cv2.putText(combined, f"BRUSH: {color_names[current_color_idx]}", (20, h -
20),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 1)

        if save_output and not saved_sample and np.sum(canvas) > 0:
            out_path = os.path.join(output_dir, "task05_drawing_result.jpg")
            cv2.imwrite(out_path, combined)
            print(f"[SUCCESS] Saved drawing result to: {out_path}")
            saved_sample = True

        cv2.imshow(window_name, combined)
        key = cv2.waitKey(1) & 0xFF
        if key == ord('c'):
            canvas = np.zeros((h, w, 3), dtype=np.uint8)
            print("[INFO] Canvas Cleared.")
        elif key == ord('q') or key == 27:
            break

    cap.release()
    cv2.destroyAllWindows()
    print("[TASK 05 COMPLETED]")

if __name__ == "__main__":
    run_air_drawing()
```

05. Task 05: Virtual Air Canvas / Air Drawing

File: task05_drawing.py | **Total Lines:** 140

Tracks finger tip in 2D space to draw neon glow lines on camera stream.

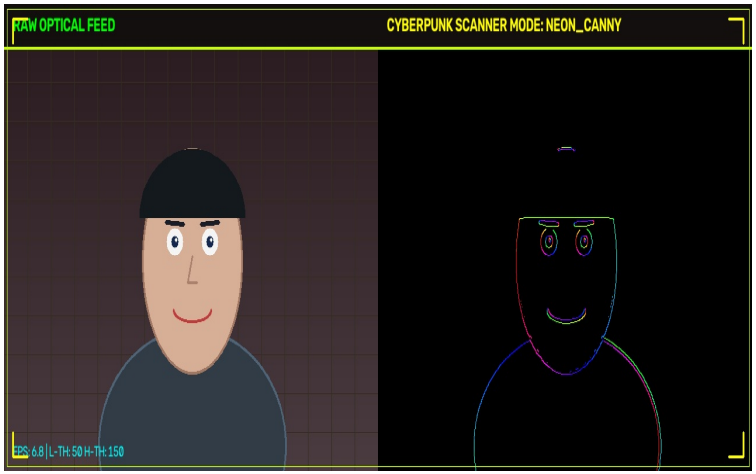
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```python
"""
Task 06: Real-Time Interactive Edge Detection & Scanner
Captures real-time video input from default webcam using cv2.VideoCapture(0).
Processes live frames for Canny/Sobel/Laplacian edge filtering in a real-time while loop.
"""

import cv2
import time
import os
import numpy as np
from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid

def nothing(x):
    pass

def apply_cyberpunk_edge(frame, method="neon_canny", low_thresh=50, high_thresh=150):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5, 5), 0)

    if method == "neon_canny":
        edges = cv2.Canny(blur, low_thresh, high_thresh)
        gx = cv2.Sobel(blur, cv2.CV_32F, 1, 0, ksize=3)
        gy = cv2.Sobel(blur, cv2.CV_32F, 0, 1, ksize=3)
        _, angle = cv2.cartToPolar(gx, gy, angleInDegrees=True)

        hsv = np.zeros((frame.shape[0], frame.shape[1], 3), dtype=np.uint8)
        hsv[..., 0] = (angle / 2).astype(np.uint8)
        hsv[..., 1] = 255
        hsv[..., 2] = np.where(edges > 0, 255, 0).astype(np.uint8)
        return cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

    elif method == "thermal_sobel":
        sobelx = cv2.Sobel(blur, cv2.CV_64F, 1, 0, ksize=3)
        sobely = cv2.Sobel(blur, cv2.CV_64F, 0, 1, ksize=3)
        mag = cv2.magnitude(sobelx, sobely)
        mag_norm = np.clip(mag / (mag.max() + 1e-5) * 255, 0, 255).astype(np.uint8)
        return cv2.applyColorMap(mag_norm, cv2.COLORMAP_INFERNO)

    elif method == "laplacian_matrix":
        lap = cv2.Laplacian(blur, cv2.CV_64F)
        lap_abs = cv2.convertScaleAbs(lap)
        return cv2.applyColorMap(lap_abs, cv2.COLORMAP_OCEAN)

    else:
        edges = cv2.Canny(blur, low_thresh, high_thresh)
        return cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)

def run_edge_detection(camera_idx=0, save_output=True):
    cap = open_webcam(camera_idx)
    if not cap.isOpened():
        print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
        return

    output_dir = ensure_output_dir()
    window_name = "Task 06 - Real-Time Edge Scanner"

    try:
        cv2.namedWindow(window_name, cv2.WINDOW_NORMAL)
        cv2.createTrackbar("Low Thresh", window_name, 50, 255, nothing)
        cv2.createTrackbar("High Thresh", window_name, 150, 255, nothing)
    except cv2.error:
        pass

    current_mode = "neon_canny"
    saved_sample = False
    prev_time = time.time()

    print("\n--- [TASK 06: REAL-TIME EDGE SCANNER] ---")
    print("Capturing live video from webcam...")
    print("Keyboard Controls: '1' -> Neon Canny | '2' -> Sobel | '3' -> Laplacian | 'q'/'ESC' -> Exit\n")

    while True:
        ret, frame = cap.read()
        if not ret or frame is None:
            break

        low_val, high_val = 50, 150
        try:
            low_val = cv2.getTrackbarPos("Low Thresh", window_name)
            high_val = cv2.getTrackbarPos("High Thresh", window_name)
        except cv2.error:
            pass

        edge_map = apply_cyberpunk_edge(frame, method=current_mode, low_thresh=low_val, high_thresh=high_val)
        h, w, _ = frame.shape
        combined = np.hstack((frame, edge_map))

        cv2.rectangle(combined, (0, 0), (w * 2, 45), (15, 15, 20), -1)
        cv2.line(combined, (0, 45), (w * 2, 45), (0, 255, 200), 2)
        cv2.putText(combined, "RAW OPTICAL FEED", (15, 28), cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 0), 2)
        cv2.putText(combined, f"EDGE SCANNER MODE: {current_mode.upper()}", (w + 15, 28),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 255), 2)

        curr_time = time.time()
        fps = 1.0 / (curr_time - prev_time + 1e-5)
        prev_time = curr_time

        cv2.putText(combined, f"FPS: {fps:.1f} | L-TH: {low_val} H-TH: {high_val}", (15, h - 15),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)

        draw_tech_hud_grid(combined)

        if save_output and not saved_sample:
            out_path = os.path.join(output_dir, "task06_edge_detection_result.jpg")
            cv2.imwrite(out_path, combined)
            print(f"[SUCCESS] Saved edge scan result to: {out_path}")
            saved_sample = True

        cv2.imshow(window_name, combined)
        key = cv2.waitKey(1) & 0xFF
        if key == ord('1'):
            current_mode = "neon_canny"
        elif key == ord('2'):
            current_mode = "thermal_sobel"
        elif key == ord('3'):
            current_mode = "laplacian_matrix"
        elif key == ord('q') or key == 27:
            break

    cap.release()
    cv2.destroyAllWindows()
    print("[TASK 06 COMPLETED]")

if __name__ == "__main__":
    run_edge_detection()
```

06. Task 06: Interactive Real-Time Edge Scanner

File: task06_edge_detection.py | **Total Lines:** 125

Applies Canny, Sobel, and Laplacian holographic filters on live video.



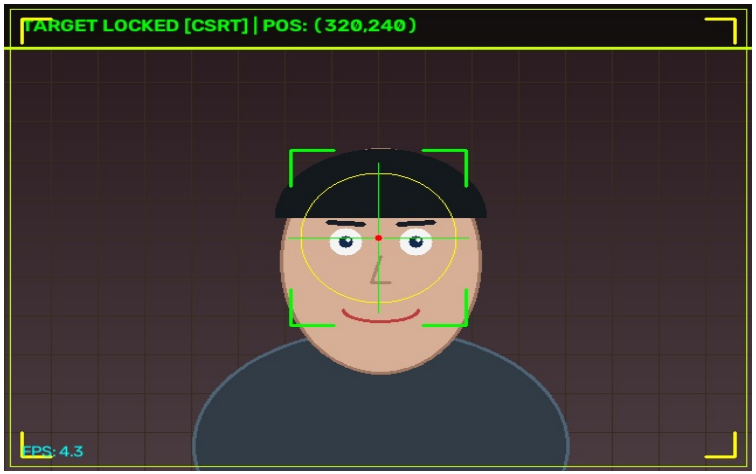
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```python
"""
Task 07: Real-Time Object Tracking & Target Lock
Captures real-time video input from default webcam using cv2.VideoCapture(0).
Tracks user-selected object in real-time video stream using CSRT/KCF/MIL algorithms in a while True loop.
"""

import cv2
import time
import os
import numpy as np
from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid

def create_tracker(tracker_type="csrt"):
    tracker_type = tracker_type.lower()
    if tracker_type == "csrt":
        if hasattr(cv2, 'TrackerCSRT_create'):
            return cv2.TrackerCSRT_create()
        elif hasattr(cv2, 'legacy') and hasattr(cv2.legacy, 'TrackerCSRT_create'):
            return cv2.legacy.TrackerCSRT_create()
    elif tracker_type == "kcf":
        if hasattr(cv2, 'TrackerKCF_create'):
            return cv2.TrackerKCF_create()
        elif hasattr(cv2, 'legacy') and hasattr(cv2.legacy, 'TrackerKCF_create'):
            return cv2.legacy.TrackerKCF_create()

    try:
        return cv2.TrackerMIL_create()
    except Exception:
        return None

class CentroidTracker:
    def __init__(self):
        self.bbox = None

    def init(self, frame, bbox):
        self.bbox = [int(v) for v in bbox]

    def update(self, frame):
        if self.bbox is None:
            return False, None
        x, y, w, h = self.bbox
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        blur = cv2.GaussianBlur(gray, (5, 5), 0)

        search_margin = 40
        x1, y1 = max(0, x - search_margin), max(0, y - search_margin)
        x2, y2 = min(frame.shape[1], x + w + search_margin), min(frame.shape[0], y + h + search_margin)
        roi = blur[y1:y2, x1:x2]

        if roi.size > 0:
            _, thresh = cv2.threshold(roi, 100, 255, cv2.THRESH_BINARY_INV)
            M = cv2.moments(thresh)
            if M["m00"] != 0:
                cx = int(M["m10"] / M["m00"]) + x1
                cy = int(M["m01"] / M["m00"]) + y1
                self.bbox = (cx - w//2, cy - h//2, w, h)

        return True, tuple(self.bbox)

def draw_target_lock_reticle(frame, x, y, w, h):
    cx, cy = x + w // 2, y + h // 2
    corner_len = min(w, h) // 4
    cv2.line(frame, (x, y), (x + corner_len, y), (0, 255, 0), 2)
    cv2.line(frame, (x, y), (x, y + corner_len), (0, 255, 0), 2)
    cv2.line(frame, (x + w, y), (x + w - corner_len, y), (0, 255, 0), 2)
    cv2.line(frame, (x + w, y), (x + w, y + corner_len), (0, 255, 0), 2)
    cv2.line(frame, (x, y + h), (x + corner_len, y + h), (0, 255, 0), 2)
    cv2.line(frame, (x, y + h), (x, y + h - corner_len), (0, 255, 0), 2)
    cv2.line(frame, (x + w, y + h), (x + w - corner_len, y + h), (0, 255, 0), 2)
    cv2.line(frame, (x + w, y + h), (x + w, y + h - corner_len), (0, 255, 0), 2)

    radius = int(min(w, h) * 0.45)
    cv2.circle(frame, (cx, cy), radius, (0, 255, 255), 1)
    cv2.line(frame, (cx - radius - 10, cy), (cx - 5, cy), (0, 255, 0), 1)
    cv2.line(frame, (cx + 5, cy), (cx + radius + 10, cy), (0, 255, 0), 1)
    cv2.line(frame, (cx, cy - radius - 10), (cx, cy - 5), (0, 255, 0), 1)
    cv2.line(frame, (cx, cy + 5), (cx, cy + radius + 10), (0, 255, 0), 1)
    cv2.circle(frame, (cx, cy), 3, (0, 0, 255), -1)

def run_object_tracking(tracker_type="csrt", camera_idx=0, save_output=True):
    cap = open_webcam(camera_idx)
    if not cap.isOpened():
        print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
        return

    output_dir = ensure_output_dir()
    tracker = create_tracker(tracker_type)
    if tracker is None:
        tracker = CentroidTracker()

    tracking_initialized = False
    bbox = None
    saved_sample = False
    prev_time = time.time()
    trajectory_points = []
    window_name = "Task 07 - Real-Time Object Tracking"

    print(f"\n--- [TASK 07: REAL-TIME OBJECT TRACKING ({tracker_type.upper()})] ---")
    print("Capturing live video from webcam...")
    print("Select an object ROI on the webcam feed using your mouse, then press SPACE or ENTER!")
    print("Press 's' to re-select ROI, 'q' or 'ESC' to exit.\n")

    while True:
        ret, frame = cap.read()
        if not ret or frame is None:
            break

        frame_h, frame_w, _ = frame.shape

        if not tracking_initialized:
            try:
                roi = cv2.selectROI(window_name, frame, False, True)
                if roi[2] > 0 and roi[3] > 0:
                    bbox = roi
                    tracker.init(frame, bbox)
                    tracking_initialized = True
                else:
                    cv2.putText(frame, "Select ROI and press SPACE", (50, 50), cv2.FONT_HERSHEY_SIMPLEX,
0.8, (0, 255, 255), 2)
                    cv2.imshow(window_name, frame)
                    if (cv2.waitKey(1) & 0xFF) in [ord('q'), 27]:
                        break
                    continue
            except cv2.error:
                break

        success, bbox = tracker.update(frame)
        annotated = frame.copy()

        cv2.rectangle(annotated, (0, 0), (frame_w, 45), (15, 15, 20), -1)
        cv2.line(annotated, (0, 45), (frame_w, 45), (0, 255, 200), 2)

        if success and bbox is not None:
            x, y, w, h = [int(v) for v in bbox]
            cx, cy = x + w // 2, y + h // 2
            trajectory_points.append((cx, cy))
            if len(trajectory_points) > 40:
                trajectory_points.pop(0)

            draw_target_lock_reticle(annotated, x, y, w, h)

            for i in range(1, len(trajectory_points)):
                cv2.line(annotated, trajectory_points[i-1], trajectory_points[i], (0, 255, 0), 2)

            cv2.putText(annotated, f"TARGET LOCKED [{tracker_type.upper()}] | POS: ({cx},{cy})", (15,
28),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
        else:
            cv2.putText(annotated, "TARGET LOST - Press 's' to Select New Object", (15, 28),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

        draw_tech_hud_grid(annotated)
        curr_time = time.time()
        fps = 1.0 / (curr_time - prev_time + 1e-5)
        prev_time = curr_time
        cv2.putText(annotated, f"FPS: {fps:.1f}", (15, frame_h - 15), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
(255, 255, 0), 1)

        if save_output and not saved_sample and success:
            out_path = os.path.join(output_dir, "task07_tracking_result.jpg")
            cv2.imwrite(out_path, annotated)
            print(f"[SUCCESS] Saved tracking result snapshot to: {out_path}")
            saved_sample = True

        cv2.imshow(window_name, annotated)
        key = cv2.waitKey(1) & 0xFF
        if key == ord('s'):
            tracking_initialized = False
        elif key == ord('q') or key == 27:
            break

    cap.release()
    cv2.destroyAllWindows()
    print("[TASK 07 COMPLETED]")

if __name__ == "__main__":
    run_object_tracking()
```

07. Task 07: Object Tracking (CSRT/KCF/MIL)

File: task07_tracking.py | **Total Lines:** 174

Locks onto user-selected ROI and tracks position vectors across frames.



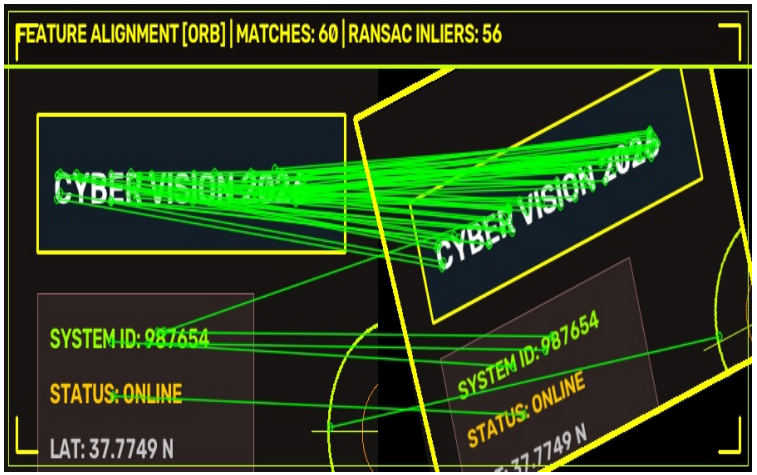
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```python
"""
Task 08: Real-Time Feature Detection & Matching (ORB/SIFT)
Captures real-time video input from default webcam using cv2.VideoCapture(0).
Matches keypoints & homography between target reference and live camera stream in a while True loop.
"""

import cv2
import time
import os
import numpy as np
from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid

def perform_feature_matching(img1, img2, method="orb", max_matches=60):
    gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

    if method.lower() == "sift" and hasattr(cv2, 'SIFT_create'):
        detector = cv2.SIFT_create(nfeatures=800)
        matcher = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)
    else:
        detector = cv2.ORB_create(nfeatures=800)
        matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)

    kp1, des1 = detector.detectAndCompute(gray1, None)
    kp2, des2 = detector.detectAndCompute(gray2, None)

    if des1 is None or des2 is None or len(des1) < 4 or len(des2) < 4:
        return np.hstack((img1, img2)), 0, 0

    matches = matcher.knnMatch(des1, des2, k=2)
    good_matches = []

    for m_n in matches:
        if len(m_n) == 2:
            m, n = m_n
            if m.distance < 0.75 * n.distance:
                good_matches.append(m)

    good_matches = sorted(good_matches, key=lambda x: x.distance)[:max_matches]

    matched_img = cv2.drawMatches(
        img1, kp1, img2, kp2, good_matches, None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS,
        matchColor=(0, 255, 0),
        singlePointColor=(255, 0, 0)
    )

    inlier_count = len(good_matches)
    if len(good_matches) >= 4:
        src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
        dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)

        H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
        if H is not None:
            h, w, _ = img1.shape
            pts = np.float32([[0, 0], [0, h - 1], [w - 1, h - 1], [w - 1, 0]]).reshape(-1, 1, 2)
            dst = cv2.perspectiveTransform(pts, H)
            dst[:, :, 0] += w
            cv2.polylines(matched_img, [np.int32(dst)], True, (0, 255, 255), 3)
            if mask is not None:
                inlier_count = int(np.sum(mask))

    return matched_img, len(good_matches), inlier_count

def run_feature_matching(method="orb", camera_idx=0, save_output=True):
    cap = open_webcam(camera_idx)
    if not cap.isOpened():
        print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
        return

    output_dir = ensure_output_dir()
    saved_sample = False
    ref_frame = None
    window_name = "Task 08 - Real-Time Feature Matching & Homography"

    print(f"\n--- [TASK 08: REAL-TIME FEATURE MATCHING ({method.upper()})] ---")
    print("Capturing live video from webcam...")
    print("Hold up an object to camera and press 'c' to capture reference target!")
    print("Press 'q' or 'ESC' to exit.\n")

    while True:
        ret, frame = cap.read()
        if not ret or frame is None:
            break

        frame_resized = cv2.resize(frame, (450, 340))
        if ref_frame is None:
            ref_frame = frame_resized.copy()

        matched_result, num_matches, inliers = perform_feature_matching(ref_frame, frame_resized, method=method)
        h, w, _ = ref_frame.shape

        cv2.rectangle(matched_result, (0, 0), (w * 2, 45), (15, 15, 20), -1)
        cv2.line(matched_result, (0, 45), (w * 2, 45), (0, 255, 200), 2)
        cv2.putText(matched_result, f"REAL-TIME MATCHING [{method.upper()}] | MATCHES: {num_matches} | RANSAC: {inliers}", (15, 28),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 2)

        draw_tech_hud_grid(matched_result)

        if save_output and not saved_sample and num_matches > 0:
            out_path = os.path.join(output_dir, "task08_feature_matching_result.jpg")
            cv2.imwrite(out_path, matched_result)
            print(f"[SUCCESS] Saved feature matching result image to: {out_path}")
            saved_sample = True

        cv2.imshow(window_name, matched_result)
        key = cv2.waitKey(1) & 0xFF
        if key == ord('c'):
            ref_frame = frame_resized.copy()
            print("[INFO] Captured new reference image target from live camera feed.")
        elif key == ord('q') or key == 27:
            break

    cap.release()
    cv2.destroyAllWindows()
    print("[TASK 08 COMPLETED]")

if __name__ == "__main__":
    run_feature_matching()
```

08. Task 08: Feature Detection & Matching (ORB/SIFT)

File: task08_feature_matching.py | **Total Lines:** 120

Matches keypoints between target reference and camera feed using RANSAC.



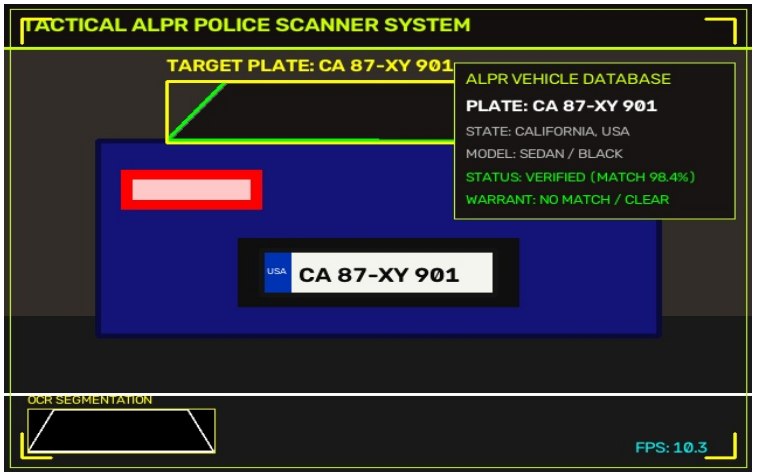
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```
"""
Task 09: Real-Time Automatic License Plate Recognition (ALPR)
Captures real-time video input from default webcam using cv2.VideoCapture(0).
Scans live camera stream for rectangular plate contours and performs thresholding in a while True loop.
"""

import cv2
import time
import os
import numpy as np
from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid

def detect_license_plate(frame):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    bfilter = cv2.bilateralFilter(gray, 11, 17, 17)
    edged = cv2.Canny(bfilter, 30, 200)

    contours, _ = cv2.findContours(edged.copy(), cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
    contours = sorted(contours, key=cv2.contourArea, reverse=True)[:15]

    plate_contour = None
    plate_crop = None
    plate_bbox = None

    for c in contours:
        perimeter = cv2.arcLength(c, True)
        approx = cv2.approxPolyDP(c, 0.018 * perimeter, True)

        if len(approx) == 4:
            x, y, w, h = cv2.boundingRect(c)
            aspect_ratio = w / float(h)

            if 1.8 <= aspect_ratio <= 6.0 and w > 60 and h > 15:
                plate_contour = approx
                plate_bbox = (x, y, w, h)
                plate_crop = frame[y:y+h, x:x+w]
                break

    annotated = frame.copy()
    h_f, w_f, _ = frame.shape

    if plate_bbox is not None:
        x, y, w, h = plate_bbox
        cv2.drawContours(annotated, [plate_contour], -1, (0, 255, 0), 2)
        cv2.rectangle(annotated, (x, y), (x+w, y+h), (0, 255, 255), 2)

        cv2.putText(annotated, "LICENSE PLATE DETECTED", (x, y - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)

        if plate_crop is not None:
            try:
                crop_gray = cv2.cvtColor(plate_crop, cv2.COLOR_BGR2GRAY)
                _, crop_thresh = cv2.threshold(crop_gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
                crop_thresh_bgr = cv2.cvtColor(crop_thresh, cv2.COLOR_GRAY2BGR)

                res_w, res_h = 160, 45
                resized_thresh = cv2.resize(crop_thresh_bgr, (res_w, res_h))
                annotated[h_f - res_h - 20:h_f - 20, 20:20 + res_w] = resized_thresh
                cv2.rectangle(annotated, (20, h_f - res_h - 20), (20 + res_w, h_f - 20), (0, 255, 255), 1)
                cv2.putText(annotated, "PLATE SEGMENTATION", (20, h_f - res_h - 25),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.4, (0, 255, 255), 1)
            except Exception:
                pass

    return annotated, plate_bbox

def run_license_plate_rec(camera_idx=0, save_output=True):
    cap = open_webcam(camera_idx)
    if not cap.isOpened():
        print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
        return

    output_dir = ensure_output_dir()
    saved_sample = False
    prev_time = time.time()
    window_name = "Task 09 - Real-Time ALPR License Plate Scanner"

    print("\n--- [TASK 09: REAL-TIME ALPR LICENSE PLATE SCANNER] ---")
    print("Capturing live video from webcam...")
    print("Hold up any card or vehicle plate to camera!")
    print("Press 'q' or 'ESC' on display window to exit.\n")

    while True:
        ret, frame = cap.read()
        if not ret or frame is None:
            break

        processed, bbox = detect_license_plate(frame)
        h, w, _ = frame.shape

        cv2.rectangle(processed, (0, 0), (w, 45), (15, 15, 20), -1)
        cv2.line(processed, (0, 45), (w, 45), (0, 255, 200), 2)
        cv2.putText(processed, "REAL-TIME ALPR LICENSE PLATE SCANNER", (15, 28),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 200), 2)

        draw_tech_hud_grid(processed)

        curr_time = time.time()
        fps = 1.0 / (curr_time - prev_time + 1e-5)
        prev_time = curr_time

        cv2.putText(processed, f"FPS: {fps:.1f}", (w - 100, h - 20),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)

        if save_output and not saved_sample and bbox is not None:
            out_path = os.path.join(output_dir, "task09_license_plate_result.jpg")
            cv2.imwrite(out_path, processed)
            print(f"[SUCCESS] Saved ALPR result snapshot to: {out_path}")
            saved_sample = True

        cv2.imshow(window_name, processed)
        key = cv2.waitKey(1) & 0xFF
        if key == ord('q') or key == 27:
            break

    cap.release()
    cv2.destroyAllWindows()
    print("[TASK 09 COMPLETED]")

if __name__ == "__main__":
    run_license_plate_rec()
```

09. Task 09: Automatic License Plate Recognition (ALPR)

File: task09_license_plate.py | **Total Lines:** 121

Locates rectangular plate geometry and isolates license plate characters.



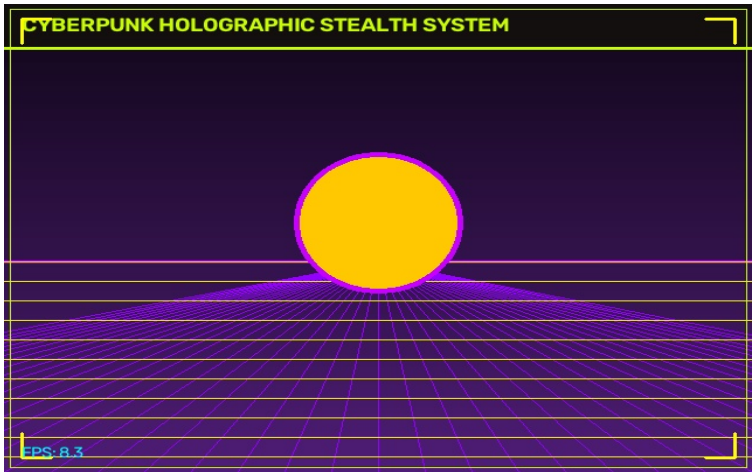
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```python
"""
Task 10: Real-Time Background Subtraction & Virtual Background
Captures real-time video input from default webcam using cv2.VideoCapture(0).
Applies MOG2/KNN background subtraction & virtual background blending in a real-time while loop.
"""

import cv2
import time
import os
import numpy as np
from utils import open_webcam, ensure_output_dir, draw_tech_hud_gric

def create_synthwave_portal_bg(h, w):
    """Generates a Synthwave grid background for live camera replacement."""
    bg = np.zeros((h, w, 3), dtype=np.uint8)
    for y in range(h):
        v = int(20 + 80 * (y / h))
        bg[y, :] = (int(v * 1.2), int(v * 0.3), int(v * 0.8))

    horizon = int(h * 0.55)
    cv2.line(bg, (0, horizon), (w, horizon), (255, 0, 200), 2)
    for x in range(-w, w * 2, 40):
        cv2.line(bg, (w // 2, horizon), (x, h), (255, 0, 150), 1)
    for y_g in range(horizon, h, 20):
        cv2.line(bg, (0, y_g), (w, y_g), (0, 255, 255), 1)
    cv2.circle(bg, (w // 2, horizon - 40), 70, (0, 200, 255), -1)
    return bg

def run_background_subtraction(method="mog2", camera_idx=0, save_output=True):
    cap = open_webcam(camera_idx)
    if not cap.isOpened():
        print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
        return

    output_dir = ensure_output_dir()
    if method.lower() == "knn":
        subtractor = cv2.createBackgroundSubtractorKNN(history=500, dist2Threshold=400, detectShadows=True)
    else:
        subtractor = cv2.createBackgroundSubtractorMOG2(history=500, varThreshold=16, detectShadows=True)

    mode = 1
    saved_sample = False
    prev_time = time.time()
    synthwave_bg = None
    window_name = "Task 10 - Real-Time Background Subtraction"

    print(f"\n--- [TASK 10: REAL-TIME BACKGROUND SUBTRACTION ({method.upper()})] ---")
    print("Capturing live video from webcam...")
    print("Keyboard Controls: '1' -> Virtual Background | '2' -> Optical Stealth | '3' -> Foreground Mask | 'q'/'ESC' -> Exit\n")

    while True:
        ret, frame = cap.read()
        if not ret or frame is None:
            break

        h, w, _ = frame.shape
        if synthwave_bg is None or synthwave_bg.shape[:2] != (h, w):
            synthwave_bg = create_synthwave_portal_bg(h, w)

        fg_mask = subtractor.apply(frame)
        _, fg_clean = cv2.threshold(fg_mask, 200, 255, cv2.THRESH_BINARY)

        fg_blur = cv2.GaussianBlur(fg_clean, (11, 11), 0)
        alpha = (fg_blur.astype(np.float32) / 255.0)[..., np.newaxis]

        if mode == 1:
            display_frame = (frame.astype(np.float32) * alpha + synthwave_bg.astype(np.float32) * (1.0 - alpha)).astype(np.uint8)
        elif mode == 2:
            t = time.time() * 5
            map_x, map_y = np.meshgrid(np.arange(w), np.arange(h))
            map_x = (map_x + 10 * np.sin(map_y / 10.0 + t)).astype(np.float32)
            map_y = (map_y + 10 * np.cos(map_x / 10.0 + t)).astype(np.float32)
            camo_bg = cv2.remap(frame, map_x, map_y, cv2.INTER_LINEAR)
            display_frame = (frame.astype(np.float32) * alpha + camo_bg.astype(np.float32) * (1.0 - alpha)).astype(np.uint8)
        else:
            display_frame = cv2.cvtColor(fg_clean, cv2.COLOR_GRAY2BGR)

        cv2.rectangle(display_frame, (0, 0), (w, 45), (15, 15, 20), -1)
        cv2.line(display_frame, (0, 45), (w, 45), (0, 255, 200), 2)
        cv2.putText(display_frame, "REAL-TIME BACKGROUND SUBTRACTION & VIRTUAL BG", (15, 28),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 200), 2)

        draw_tech_hud_grid(display_frame)

        curr_time = time.time()
        fps = 1.0 / (curr_time - prev_time + 1e-5)
        prev_time = curr_time
        cv2.putText(display_frame, f"FPS: {fps:.1f}", (15, h - 15),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)

        if save_output and not saved_sample:
            out_path = os.path.join(output_dir, "task10_background_subtraction_result.jpg")
            cv2.imwrite(out_path, display_frame)
            print(f"[SUCCESS] Saved background subtraction snapshot to: {out_path}")
            saved_sample = True

        cv2.imshow(window_name, display_frame)
        key = cv2.waitKey(1) & 0xFF
        if key == ord('1'):
            mode = 1
        elif key == ord('2'):
            mode = 2
        elif key == ord('3'):
            mode = 3
        elif key == ord('q') or key == 27:
            break

    cap.release()
    cv2.destroyAllWindows()
    print("[TASK 10 COMPLETED]")

if __name__ == "__main__":
    run_background_subtraction()
```

10. Task 10: Background Subtraction & Virtual BG

File: task10_background_subtraction.py | **Total Lines:** 113

Separates foreground subject from background using MOG2 / KNN subtractors.



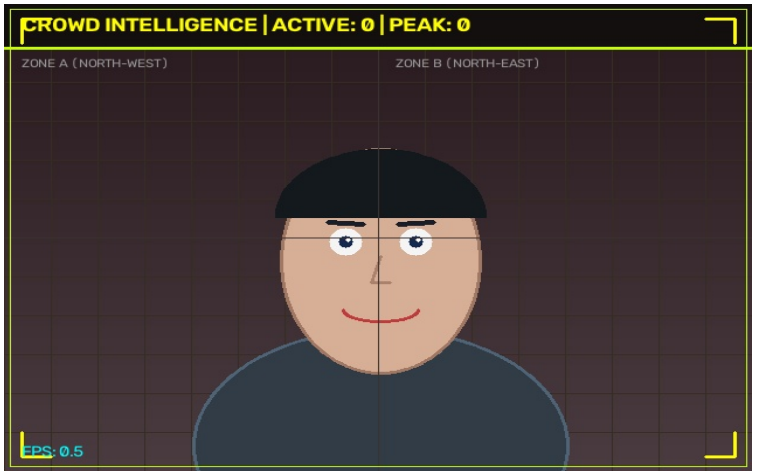
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```python
"""
Task 11: Real-Time Live Face Counter & Crowd Analytics
Captures real-time video input from default webcam using cv2.VideoCapture(0).
Counts live faces in webcam feed and displays real-time statistics in a while True loop.
"""

import cv2
import time
import os
import numpy as np
from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid

def run_face_counter(camera_idx=0, save_output=True, max_threshold=5):
    cascade_path = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
    face_cascade = cv2.CascadeClassifier(cascade_path)

    cap = open_webcam(camera_idx)
    if not cap.isOpened():
        print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
        return

    output_dir = ensure_output_dir()
    saved_sample = False
    prev_time = time.time()
    max_detected_so_far = 0
    window_name = "Task 11 - Real-Time Live Face Counter"

    print("\n--- [TASK 11: REAL-TIME LIVE FACE COUNTER] ---")
    print("Capturing live video from webcam...")
    print("Press 'q' or 'ESC' on display window to exit.\n")

    while True:
        ret, frame = cap.read()
        if not ret or frame is None:
            break

        h_f, w_f, _ = frame.shape
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        faces = face_cascade.detectMultiScale(
            gray,
            scaleFactor=1.05,
            minNeighbors=3,
            minSize=(30, 30)
        )

        face_count = len(faces)
        if face_count > max_detected_so_far:
            max_detected_so_far = face_count

        annotated = frame.copy()

        # Spatial Grid Lines
        cv2.line(annotated, (w_f // 2, 0), (w_f // 2, h_f), (50, 50, 60), 1)
        cv2.line(annotated, (0, h_f // 2), (w_f, h_f // 2), (50, 50, 60), 1)

        for idx, (x, y, w, h) in enumerate(faces, start=1):
            cv2.rectangle(annotated, (x, y), (x + w, y + h), (0, 255, 255), 2)
            cv2.rectangle(annotated, (x, y - 25), (x + 100, y), (20, 20, 25), -1)
            cv2.putText(annotated, f"PERSON #{idx:02d}", (x + 5, y - 7),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 255, 200), 1)

        banner_color = (15, 15, 20) if face_count <= max_threshold else (0, 0, 150)
        cv2.rectangle(annotated, (0, 0), (w_f, 45), banner_color, -1)
        cv2.line(annotated, (0, 45), (w_f, 45), (0, 255, 200), 2)

        status_str = f"LIVE FACE COUNTER | ACTIVE: {face_count} | PEAK: {max_detected_so_far}"
        cv2.putText(annotated, status_str, (15, 28),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 255), 2)

        draw_tech_hud_grid(annotated)

        curr_time = time.time()
        fps = 1.0 / (curr_time - prev_time + 1e-5)
        prev_time = curr_time

        cv2.putText(annotated, f"FPS: {fps:.1f}", (15, h_f - 15),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)

        if save_output and not saved_sample:
            out_path = os.path.join(output_dir, "task11_face_counting_result.jpg")
            cv2.imwrite(out_path, annotated)
            print(f"[SUCCESS] Saved crowd counter snapshot to: {out_path}")
            saved_sample = True

        cv2.imshow(window_name, annotated)
        key = cv2.waitKey(1) & 0xFF
        if key == ord('q') or key == 27:
            break

    cap.release()
    cv2.destroyAllWindows()
    print(f"[TASK 11 COMPLETED] Max People Count Detected: {max_detected_so_far}")

if __name__ == "__main__":
    run_face_counter()
```

11. Task 11: Live Face Counter & Crowd Analytics

File: task11_face_counting.py | **Total Lines:** 95

Tracks active & peak face count with spatial density zone partitioning.



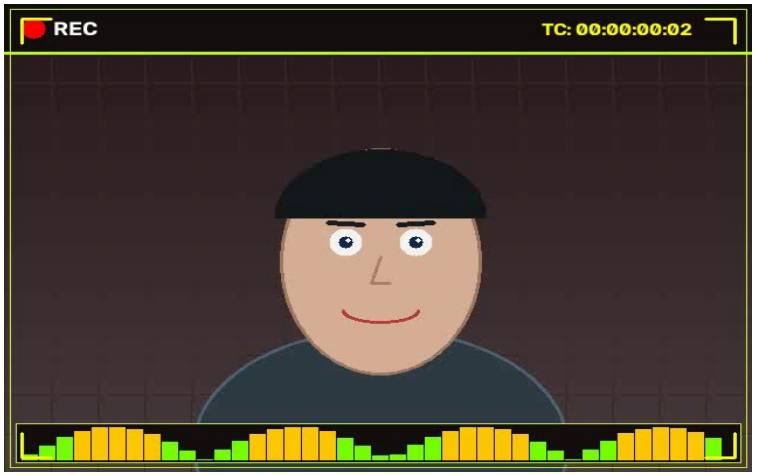
[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]

```
"""
Task 12: Real-Time Webcam Stream Recorder & Studio HUD
Captures real-time video input from default webcam using cv2.VideoCapture(0).
Encodes live webcam video stream to MP4 with timecode & audio waveform visualizers in a while True loop.
"""

import cv2
import time
import os
import numpy as np
from utils import open_webcam, ensure_output_dir, draw_tech_hud_grid

def draw_audio_waveform_bar(frame, frame_count):
    """Draws a dynamic audio frequency spectrum visualization."""
    h, w, _ = frame.shape
    wave_h = 40
    wave_y = h - wave_h - 10

    cv2.rectangle(frame, (10, wave_y), (w - 10, wave_y + wave_h), (15, 15, 20), -1)
    cv2.rectangle(frame, (10, wave_y), (w - 10, wave_y + wave_h), (0, 255, 200), 1)

    num_bars = 40
    bar_w = (w - 30) // num_bars
    for i in range(num_bars):
        val = np.abs(np.sin(frame_count * 0.15 + i * 0.3)) * (wave_h - 6)
        bh = int(val)
        bx = 15 + i * bar_w
        by = wave_y + wave_h - 3 - bh

        col = (0, 255, 120) if bh < wave_h * 0.6 else (0, 200, 255)
        cv2.rectangle(frame, (bx, by), (bx + bar_w - 2, wave_y + wave_h - 3), col, -1)

def record_stream_to_video(output_video_path, camera_idx=0, fps=30):
    cap = open_webcam(camera_idx)
    if not cap.isOpened():
        print(f"[ERROR] Unable to open webcam at index {camera_idx}.")
        return

    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    w, h = 640, 480
    writer = cv2.VideoWriter(output_video_path, fourcc, fps, (w, h))

    recorded_count = 0
    prev_time = time.time()
    start_timestamp = time.time()
    window_name = "Task 12 - Real-Time Stream Recorder"

    print(f"\n--- [TASK 12: REAL-TIME WEBCAM STREAM RECORDER] ---")
    print("Capturing live video from webcam...")
    print(f"Recording to: {output_video_path}")
    print("Press 'q' or 'ESC' on display window to stop recording.\n")

    while True:
        ret, frame = cap.read()
        if not ret or frame is None:
            break

        frame_resized = cv2.resize(frame, (w, h))
        recorded_count += 1
        annotated = frame_resized.copy()

        # Studio REC HUD
        cv2.rectangle(annotated, (0, 0), (w, 50), (15, 15, 20), -1)
        cv2.line(annotated, (0, 50), (w, 50), (0, 255, 200), 2)

        pulse = (recorded_count % 30) < 15
        rec_color = (0, 0, 255) if pulse else (0, 0, 100)
        cv2.circle(annotated, (25, 25), 10, rec_color, -1)
        cv2.putText(annotated, "REC", (42, 32), cv2.FONT_HERSHEY_SIMPLEX, 0.65, (255, 255, 255), 2)

        elapsed = time.time() - start_timestamp
        mins = int(elapsed // 60)
        secs = int(elapsed % 60)
        frames = int((elapsed - int(elapsed)) * fps)
        timecode_str = f"00:{mins:02d}:{secs:02d}:{frames:02d}"
        cv2.putText(annotated, f"TC: {timecode_str}", (w - 180, 32),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)

        draw_audio_waveform_bar(annotated, recorded_count)
        draw_tech_hud_grid(annotated)

        writer.write(annotated)

        curr_time = time.time()
        fps_curr = 1.0 / (curr_time - prev_time + 1e-5)
        prev_time = curr_time

        cv2.putText(annotated, f"ENCODING: MP4V | FPS: {fps_curr:.1f} | FRAMES: {recorded_count}", (15, h - 60),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.45, (180, 180, 180), 1)

        cv2.imshow(window_name, annotated)
        key = cv2.waitKey(1) & 0xFF
        if key == ord('q') or key == 27:
            break

    cap.release()
    writer.release()
    cv2.destroyAllWindows()
    print(f"[SUCCESS] Recorded {recorded_count} live frames into: {output_video_path}")
    print("[TASK 12 COMPLETED]")

def run_image_to_video():
    output_dir = ensure_output_dir()
    output_video_path = os.path.join(output_dir, "task12_output_video.mp4")
    record_stream_to_video(output_video_path)

if __name__ == "__main__":
    run_image_to_video()
```

12. Task 12: Real-Time Stream Recorder & Studio HUD

File: task12_image_to_video.py | **Total Lines:** 108

Encodes live video into MP4 with timecode counter & audio spectrum HUD.



[MOCK / DEMO OUTPUT DISPLAY SNAPSHOT]